Test Time Compute Backup Speaking Script

Additional explanations for questions and discussion

Slide 20 Constructing Revision Training Data

The revision model needs examples of how earlier answers can lead to a corrected answer. The authors independently sample sixty-four full solutions to a training question at a higher temperature.

For each correct target, they select zero to four incorrect answers from the same set as context. The number is sampled uniformly from those five options. Character edit distance helps prioritize incorrect answers related to the correct one.

The small equation example shows the purpose. Both solutions reach two x equals four. The incorrect reference then says x equals four. The target keeps the valid setup and changes the last calculation to x equals two. The model learns to use earlier work when producing a new solution.

These are training pairs assembled after independent sampling. During inference, the context window simply keeps the most recent four answers in order. Edit distance is used to build the training examples.

Slide 21 Why the Last Revision Can Be Wrong

[Point to the correct middle answer, then the incorrect final revision.]

Training contexts contain incorrect reference answers followed by a correct target. During inference, the earlier answers are generated by the revision model itself, so they can already be correct.

This difference creates a distribution shift. A model may keep revising a solution that was already right. The diagram illustrates a wrong answer becoming correct and then becoming wrong again. It is a schematic example.

Under the authors' naive revision approach, about thirty-eight percent of correct answers were converted to incorrect answers. The denominator is correct answers, rather than every generated answer.

This explains the use of final selection from the history. A verifier or majority voting gives the system a chance to recover a correct answer produced earlier. Selection itself can still make mistakes.

Slide 22 A Complete Beam Search Example

[Walk through one round with six candidates, then repeat the same rule.]

Here is the numerical beam-search procedure used in our earlier overview. Let N be six and M be two. First sample six possible first steps. Score the entire prefix of each with the trained PRM.

Keep the highest-scoring N divided by M prefixes, which is three. From each retained prefix, sample two next steps. We now have six longer candidates again.

Score all six together and repeat. Two descendants of the same prefix can both survive. The method is selecting across the whole pool of candidates. It continues until solutions finish or the maximum of forty expansion rounds is reached.

The final candidates are evaluated using weighted best-of-N. Their matching final answers receive summed scores, and one final answer is returned. N is the size of the candidate pool, while M controls how many continuations each retained prefix receives in this implementation.

Slide 23 Difficulty Estimation and Policy Selection

Difficulty is defined relative to the base model. For each test question, the authors independently sample 2,048 full answers.

Oracle difficulty uses ground-truth checking to estimate the probability that one sampled answer is correct. The questions are ranked by this rate and split into five quantiles, with about one fifth in each group.

Predicted difficulty uses the same sample set but replaces correctness labels with learned verifier scores. The mean final-answer score supplies the ranking. The procedure therefore creates a separate set of five groups. A question can fall in different groups under the two definitions.

For each group and budget, the authors choose the best tested setting and evaluate it using held-out folds. This approximates an ideal per-question policy. The 2,048 samples estimate difficulty and their cost is excluded from the plotted search or revision budgets. This matters for practical efficiency.

Slide 24 Pretraining and Inference at Equal Total FLOPs

To answer that, we must compare total computation. Training a larger model costs more, and using it to generate answers also costs more.

Let P be the small model's parameter count. Its pretraining cost is approximately six times P times the number of training tokens. Its inference cost is approximately two times P times the total inference tokens. We call these costs X and Y.

If parameters grow by a factor M, both costs grow by M. The total is M times the sum of X and Y.

The small model still spends X on training. We choose its inference multiplier a so that X plus aY equals the larger model's total cost. This equation sets an equal budget.

Slide 25 Using R to Compare Total Compute

R is the ratio of total baseline inference tokens to pretraining tokens. We use this ratio for the cost comparison. R is different from a, the small model's inference budget multiplier.

The small-model curve tells us the accuracy at each inference budget. R tells us which budget has the same total cost as the larger model. This total includes pretraining and inference.

Start with the equal-cost equation from the previous slide. Solve for a, replace X and Y with their cost formulas, and cancel the parameter count. Six divided by two gives three. The training-to-inference token ratio is one divided by R. This gives the final formula.

For one fixed R, only the matched value of a gives equal total cost. If we give the small model more compute than this, its total cost becomes higher. These equations give the equal-cost comparison points in the main result figure.

The paper considers three R values. R of twenty-two gives roughly sixteen times the small model's baseline inference budget. R of zero point seven nine gives about sixty-four times. R of zero point one six gives about two hundred and fifty-six times.

These three budgets give the three equal-cost positions in Figure 9. The larger model still gives a greedy answer. We compare its accuracy with the small-model curve at each position.